

SYSTEM INTEGRATION PRACTICES



Integration Styles

Instructor: Huy Nguyen Dang Quang, Msc

Email: huyndq@duytan.edu.vn

REF: Shawn A. Butler, Ph.D

Senior Lecturer, Executive Education Program

Institute for Software Research

Carnegie Mellon University

Lecture Objectives

- Be able to identify the four different styles of application integration
- Understand the elements of a messaging system
- Become familiar with some of the problems associated with senders and receivers

- The application developers may not have designed with integration in mind
- Even if they did, they probably wouldn't have anticipated your requirements
- The documentation is perfectly understandable in the minds of the original developers
- The interface documents long ago disappeared

Integration Guidelines

- Application Coupling – Minimize dependencies
 - Tightly coupled applications have known and unknown assumptions
 - Applications can evolve independently without problems
- Intrusiveness – Minimize changes in each application
 - Changes are often necessary
 - Tradeoff between intrusiveness and best integration design

Integration Guidelines

- Technology Selection – A variety of HW and SW integration tools
 - Expensive
 - Vendor Lock-in
 - Tradeoff between tools and reinventing
- Data Format – Agree on format of data exchange
 - Change the application or create a translator
 - How will the data change over time?
- Data timeliness – When does the exchange take place
 - Tradeoff large chunks versus small chunks of data
 - Real time - near real time – in time
- **Data format transformation**
 - **Apache Kafka Streams or Azure Data Factory.**

Integration Guidelines

- Data or Functionality – functionality provides better abstraction between applications
 - Local versus remote invocation
- Remote Communication
 - Synchronous and Asynchronous
 - Slower than local
- Reliability – Communication across networks is not as reliable as inter-application communication

Application Integration Options

There's more than one approach for integrating applications. Each approach addresses some of the integration criteria better than others. The various approaches can be summed up in **four main integration styles**:

File Transfer: Have each application produce files of shared data for others to consume, and consume files that others have produced.

Shared Database: Have the applications store the data they wish to share in a common database.

Remote Procedure Invocation: Have each application expose some of its procedures so that they can be invoked remotely, and have applications invoke those to run behavior and exchange data.

Messaging: Have each application connect to a common messaging system, and exchange data and invoke behavior using messages.

File Transfer is the process of **sending, receiving, or sharing files between systems, devices, or users** through network protocols or storage methods. It is a simple and widely used way to transfer data without requiring direct system integration.

*The **File Transfer** process typically involves the following steps:*

1. **File Creation** – The source system generates a file containing the necessary data (CSV, XML, JSON, Excel, etc.).
2. **File Transmission** – The file is sent from the source system to the destination system using methods such as FTP, SFTP, HTTP, Email, or cloud storage.
3. **File Reception & Storage** – The destination system receives the file and stores it in a designated folder or database.

Common File Transfer Methods

- **FTP (File Transfer Protocol)** – A protocol for transferring files over a network.
- **SFTP (Secure File Transfer Protocol)** – A secure version of FTP.
- **HTTP/HTTPS Download** – Downloading files via a browser or API.
- **Email Attachment** – Sending files via email.
- **Cloud Storage (Google Drive, Dropbox, AWS S3, Azure Blob)** – Sharing files through cloud services.
- **Peer-to-Peer (P2P) Transfer** – Direct file transfers between devices, e.g., BitTorrent.

Examples of File Transfer

- **Accounting & Banking Systems**
 - A company exports an **Excel report of transactions** and sends it via **SFTP** to a bank for reconciliation.
- **ERP & Logistics Systems**
 - An ERP system exports a list of orders in **CSV format** and transfers it to the warehouse system via **FTP** for processing.
- **Email Attachments**
 - An accountant sends invoices to customers via email with a **PDF attachment**.

When to Use File Transfer?

- When a system does not have an API for direct connection.
- When sending batch data at scheduled intervals.
- When exchanging data between different systems without major modifications.

Examples:

- A business sends daily sales reports via **FTP** to a partner.
- A shipping company exports order data in **XML format** and uploads it to **AWS S3** for warehouse processing.

Conclusion: File Transfer is a widely used technique for exchanging data between systems in a simple way without deep integration. However, it is not the best solution for real-time data integration, so businesses should carefully evaluate its suitability before implementation.

- People buy packages that are developed by outside organizations
- Different system are built at different times, leading to different technology choices
- Different systems are built by different people, whose experience and preferences lead them to different approaches to building applications
- Getting an application out and delivering value is more important than ensuring that integration is addressed, especially when that integration isn't adding any value to the application under development.

File Integration is a method of integrating data between systems by **reading, processing, transforming, and synchronizing data from files**. This approach allows systems to **share information** without requiring direct connections (such as APIs or a shared database).

The File Integration process typically involves the following steps:

1. **File creation:** The source system exports data to a file (CSV, XML, JSON, Excel, etc.).
2. **File transfer:** The file is sent from the source system to the destination system via FTP, SFTP, Cloud Storage, Email, or API.
3. **File processing:** The destination system receives the file, reads its content, extracts data, and transforms it (ETL – Extract, Transform, Load).
4. **Data storage:** The processed data is saved into a database or the target application.

Examples of File Integration

❑ HR and Payroll System Integration

- The **HRM system** exports the employee list as a **CSV file**.
- The file is sent to the **Payroll system** via **SFTP**.
- The Payroll system **reads, transforms**, and imports the data into the **database** for salary calculations.

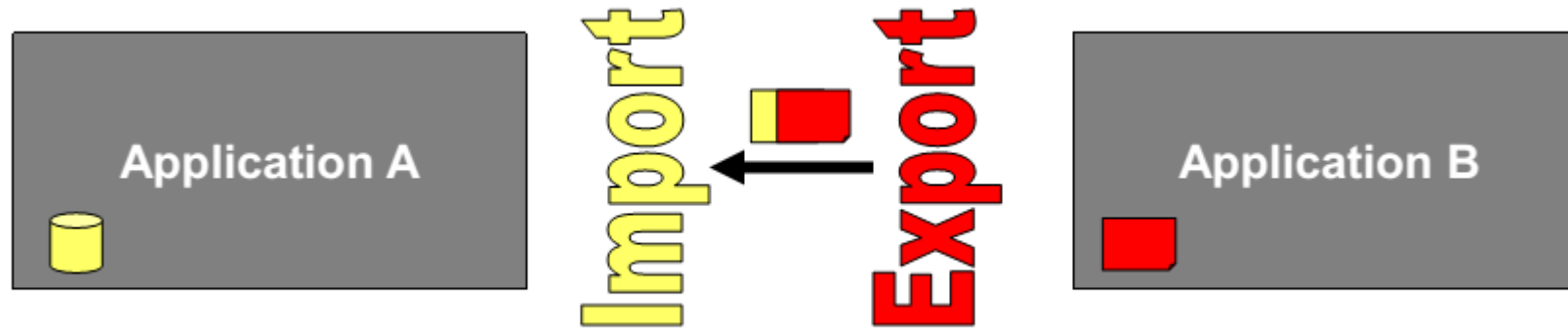
❑ ERP and Financial System Integration

- The ERP system exports order data as an **XML file**.
- The file is stored in **AWS S3**, and the financial system **downloads, processes, and imports the data**.

Technologies in File Integration

- **FTP/SFTP** – Secure file transfer between systems.
- **ETL (Extract, Transform, Load)** – SSIS, Talend, Apache NiFi.
- **Cloud Storage** – AWS S3, Azure Blob, Google Cloud Storage.
- **Middleware** – MuleSoft, Boomi, Apache Camel for automation.

Integration Styles - File Integration



- Advantages
 - Almost all applications use/produce files
 - Standard formats often available
 - No knowledge of the internal application
 - No special tools necessary
 - Receiving application can manipulate file data
- Disadvantages
 - Integrator workload
 - Formats
 - File management
 - Freshness/staleness
 - Timing and locking mechanisms
 - Bad data
 - Update latency
 - Computing resources

When to Use File Integration?

- When legacy systems do not support APIs.
- When integrating multiple incompatible systems.
- When periodic data exchange (batch processing) is required.

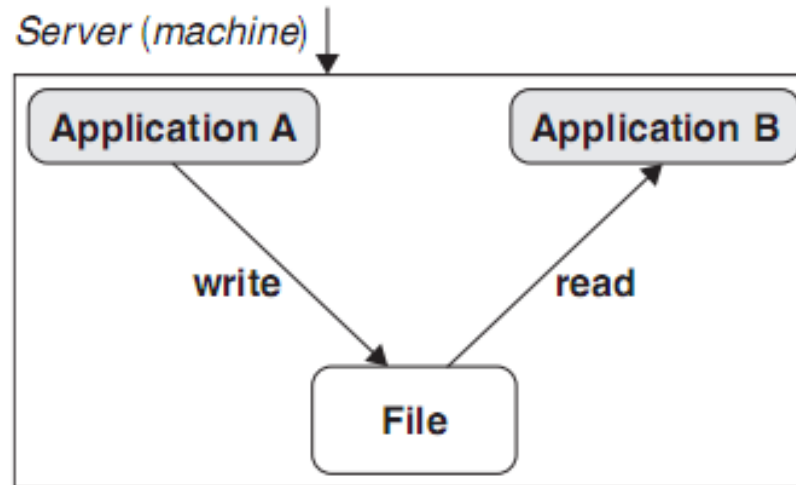
Examples:

- A bank receives daily transaction reports via **Excel files**.
- An insurance system processes customer data from **CSV files**.

Conclusion: File Integration is a widely used solution for integrating data between systems when direct API connections are not available, allowing businesses to exchange data efficiently without modifying existing systems.

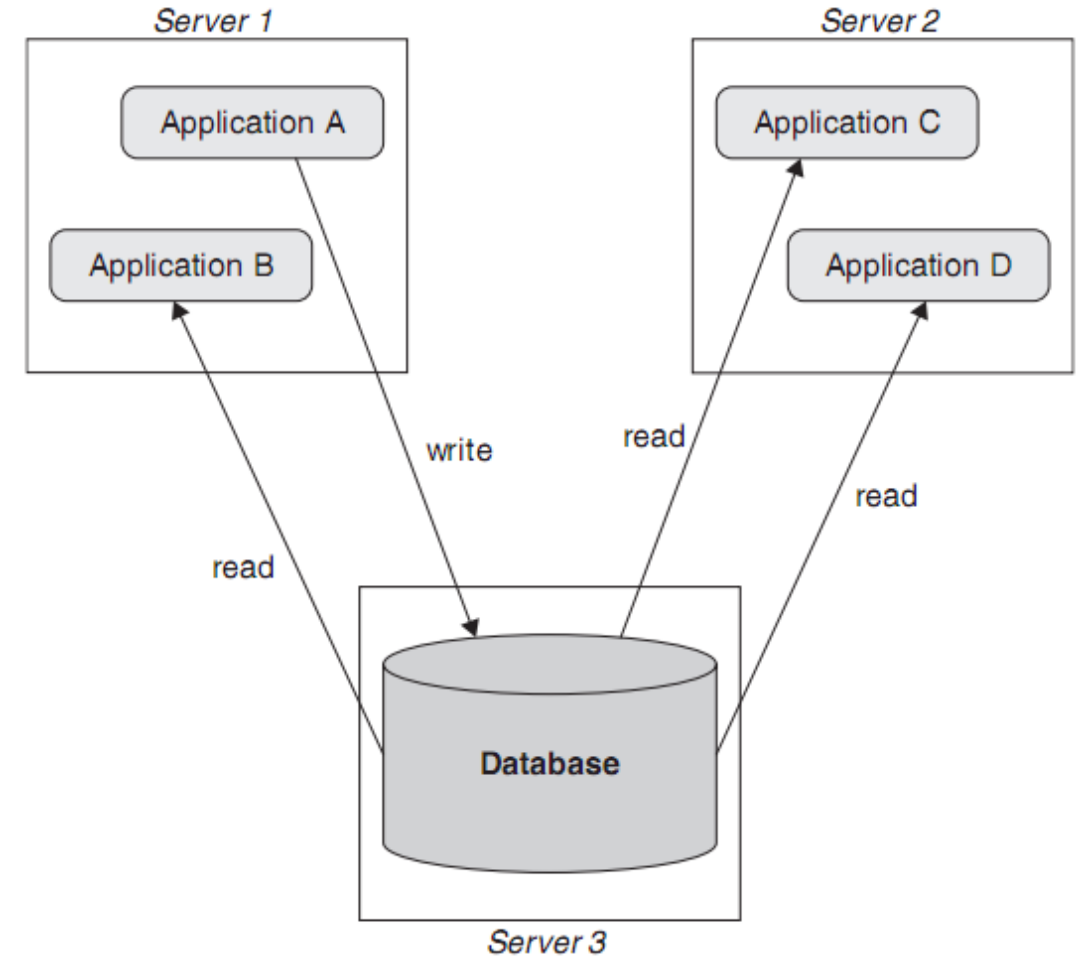
File-based Data Sharing

- In this method, one application writes data to a file, and another application reads data from that file.
- If both applications are on the same computer, they can use the hard drive to write and read the data.



Shared Database

- One application writes data to the database, and other applications read data from that database.



Integration Styles - Shared Databases

Advantages

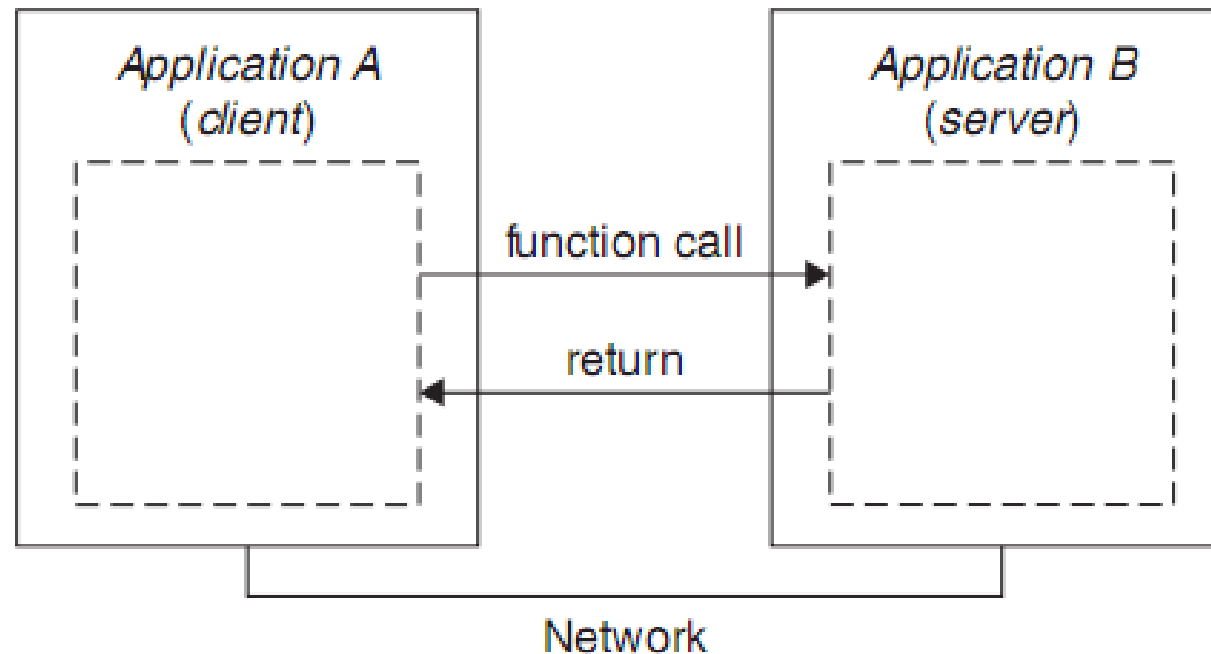
- Consistency
- Standard Query Language (SQL)
- No multiple file formats
- Single technology
- Semantic Dissonance resolved

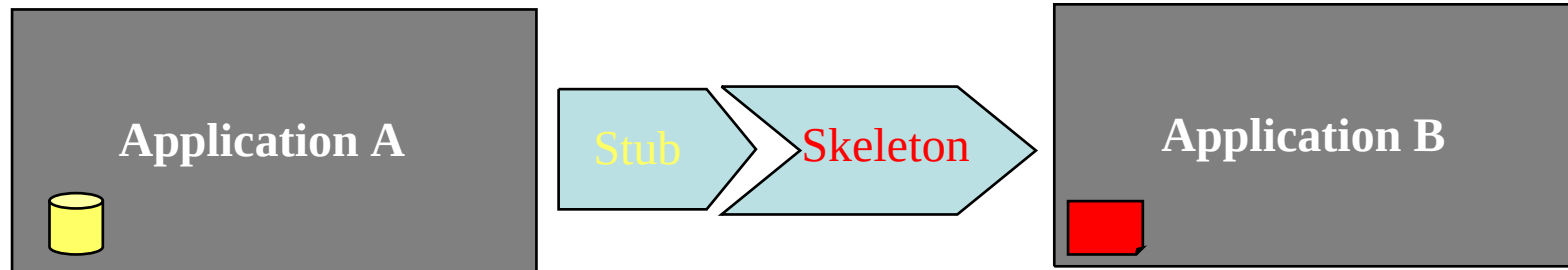
Disadvantages

- Semantic Dissonance hard to resolve
- Finding a workable schema difficult
- Changed schemas break COTS applications
- Database may become performance bottleneck
- Database changes impact performance
- Not-so-standard SQL
- Ripple effect of changes

- An enterprise has multiple applications that are being built independently, with different languages and platforms. The enterprise needs to share data and processes in a responsive way.
- **How can I integrate multiple applications so that they work together and can exchange information?**

- **RPC** is also known as “**client/server**” architecture and “**two-tier**” architecture based on socket programming.
- RPC is used between any two applications to share functionality.





How to Integrate Two (or more) Applications – RPC

A stub is typically located on the client machine, not on the server. Its role is to package the following information:

- The identifier of the remote object that needs to be used
- A description of the method to be invoked
- Encoding the parameters and sending them to the Skel.

The stub will transfer this information to the server. On the server side, the Skel object performs the following tasks to invoke the method remotely:

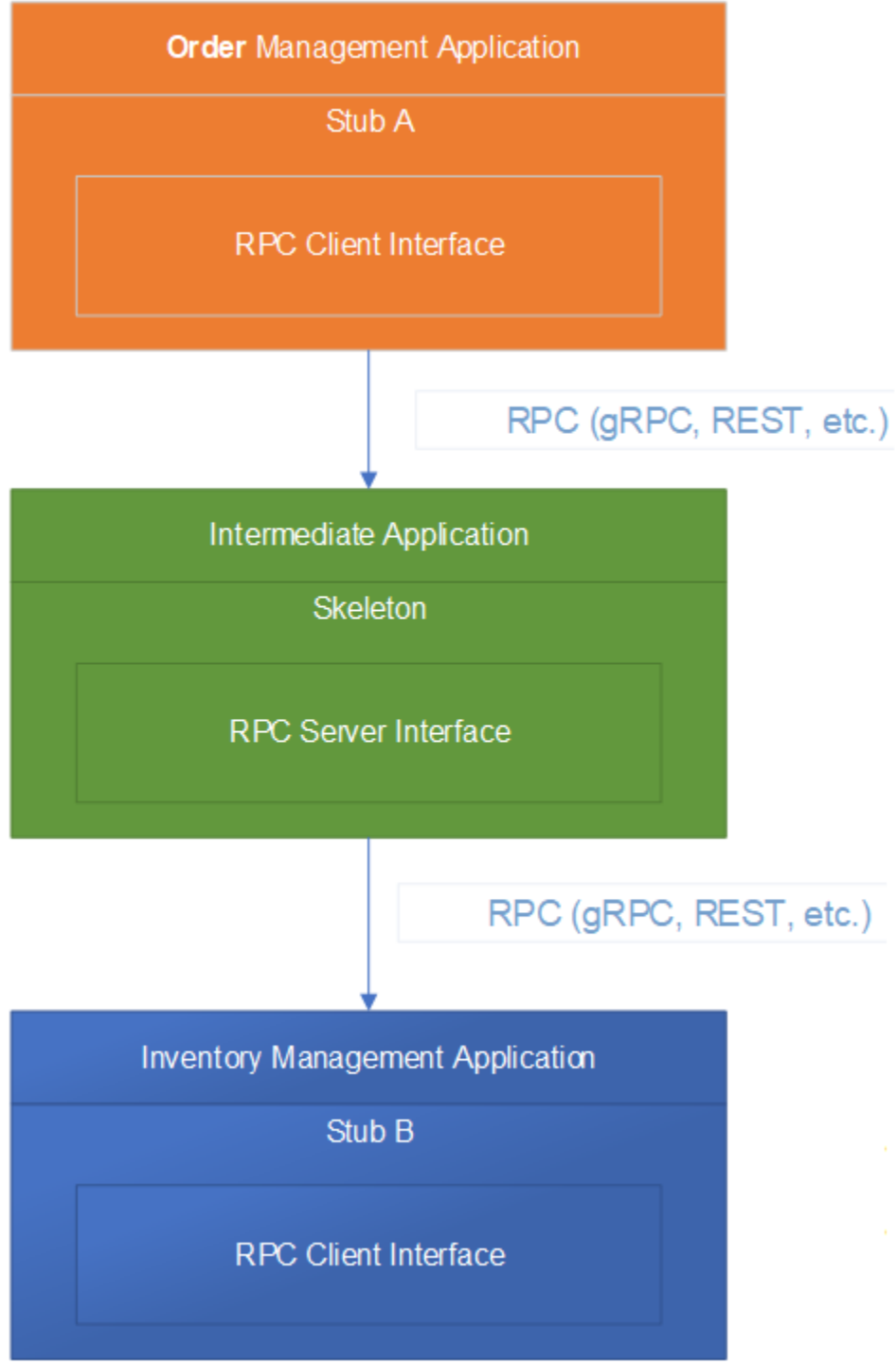
- Decode the parameters
- Identify the object to execute the corresponding function call
- Execute the method call as requested
- Gather results to respond or report any exceptions
- Send the response packaging the result data back to the stub on the client machine

Example: Order Management System

Suppose we are building an order management system for an online store. This system consists of two applications:

1. **Application A (Order Management System):** This application is used to process and manage customer orders. It stores information about the orders, such as the product list, shipping address, and order status.
2. **Application B (Inventory Management System):** This application is used to manage the store's inventory. It stores information about the quantity of products available in stock.

An **Intermediate Application** needs to be developed to transfer data from Application A to Application B when a new order is created. This intermediate application will use RPC to send requests to update the product quantity in inventory from Application A to Application B.



Invocation

- Advantages
 - Data changes can trigger other changes
 - Many existing tools
 - Easier to deal with semantic dissonance
 - Applications are less coupled than with Data Sharing style
- Disadvantages
 - Co-applications have to negotiate exchange interface
 - Interfaces evolve and grow

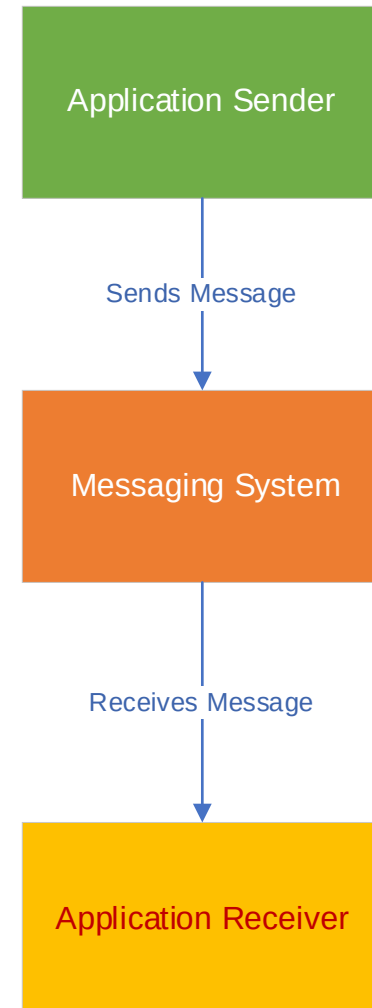
- Messaging Systems are a type of architecture or software used to facilitate communication between various components within a distributed system. Specifically, messaging systems enable applications or services to send and receive messages or data across a network or other communication channels.
- Components of a messaging system include:
 - 1. Message Queue:** This is a part of the system used to store messages before they are processed. Messages are typically placed into a queue by the sender and retrieved for processing by the receiver in a sequential or specific order.
 - 2. Message Broker:** This is a component that manages and distributes messages between different applications or services. Message brokers often handle routing, forwarding, storage, and security of messages.
 - 3. Messaging Protocol:** These are rules and standards used to format, transmit, and process messages. Common protocols in messaging systems include AMQP (Advanced Message Queuing Protocol), MQTT (Message Queuing Telemetry Transport), and STOMP (Simple Text Oriented Messaging Protocol).

- **Channels – Logical addresses**
 - Different logical addresses for different purposes
 - How does the application find the relevant addresses?
- **Messages**
 - Headers – type of data, origin, destination...
 - Body – the data
- **Pipes and Filters** – preprocesses data
- **Routing** – broadcast, point to point, context based, publish subscribe...
- **Transformation**
 - Aggregation
 - Data types
 - Data Representations (XML, ASCII..)
 - Transport (HTTP, SOAP, JMS...)
- **Endpoints** – Message endpoints interface between the application and the messaging system

How to Integrate Two (or more) Applications – Messaging

To illustrate the integration of two applications using a messaging system, let's consider a simple example of sending a notification from a sender application to a receiver application through a messaging system.

In this example, we will use two simple applications: the Send Message application and the Receive Message application.



Example: An application integrates two other applications to use a messaging system: a task management application or a personal calendar combined with a messaging app like Messenger or WhatsApp. Users can create and manage task lists, schedules, and communicate directly with team members through messages.

Features:

- **Task and Schedule Management:** Users can create, edit, and manage tasks and events in their schedules.
- **Messaging System Integration:** The system integrates with a messaging application like Messenger or WhatsApp to allow users to communicate directly with members in their task lists or schedules.
- **Notifications and Reminders:** The application sends notifications and reminders to users about upcoming tasks or events in their schedules.
- **Task Sharing:** Users can share tasks or events from their task lists or schedules with other members through messages.

Integration Styles - Messenger-Oriented Middleware (MOM)

MOM is a type of middleware specifically designed to support the sending and receiving of messages between different applications and systems. The primary goal of MOM is to provide a flexible, reliable, and distributed messaging mechanism among various software components in a distributed system. MOM is commonly used in applications that require scalability, high reliability, and availability.

Some features and benefits of MOM include:

- **Message Distribution:** MOM allows messages to be sent from one application or component and received at multiple locations within the distributed system.
- **Reliability:** MOM typically provides mechanisms to ensure that messages are sent and received reliably, using protocols like TCP or similar protocols.
- **Queue Management:** MOM often supports message queues to handle high loads and alleviate pressure on the receiving applications.
- **Message Forwarding:** MOM can forward messages between unrelated applications, helping to create a flexible and scalable distributed system.
- **Logging:** Some MOM systems provide the capability to log message sending and receiving activities for analysis and monitoring.

Integration Styles - Messenger-oriented middleware (MOM)

Online Banking System:

In an online banking system, there are various components such as account management systems, payment transaction systems, transaction notification systems, user management systems, etc. These applications need to communicate with each other to perform functions such as transferring money, paying bills, checking balances, and providing transaction information to users.

MOM can be used in this system to:

- **Send Transaction Notifications:** When a transaction is executed, the account management system can send a message to the transaction notification system to inform users about the new transaction.
- **Confirm Transactions:** The payment transaction system can send a confirmation message to the account management system when a payment transaction is successfully completed.
- **Handle Multi-tier Transactions:** MOM can support the processing of multi-tier transactions, where a transaction may need to be handled by multiple different applications within the system.
- **Manage Queues:** MOM can help manage queues of transactions waiting to be processed, enhancing system performance and reducing the risk of data loss.

Messaging Example: MegaMoney Bancorp

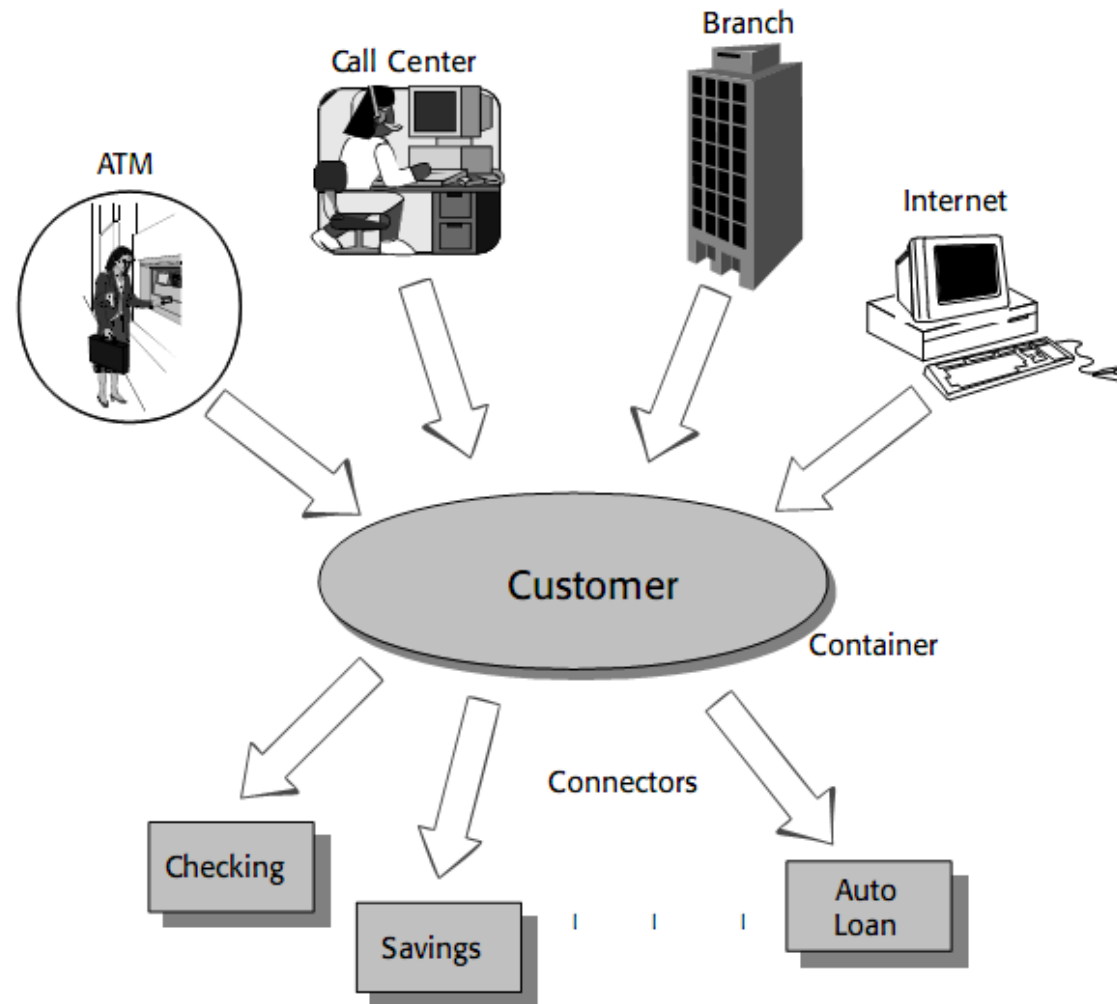


Figure 4.9 The connector-container model enables centralized access to back-end systems.

- Advantages
 - Asynchronous data transfer
 - Decoupled applications
 - Choice among topologies
 - Applications have different conceptual models
 - Timeliness
 - Reliability
- Disadvantages
 - Asynchronous learning curve
 - Testing and debugging more difficult
 - Semantic dissonance still there

- System integration would be easier if applications were designed to be integrated, had similar formats, and had complete and understandable documentation
- Each integration style has advantages and disadvantages
- Messaging systems have become the preferred method of integration, but it isn't the only method of integration
- Messaging products provide the foundation, but there is still a lot of design to be done

The end

THANK YOU